



Universidad Andrés Bello
Facultad de Ingeniería

Tarea 4

**Análisis comparativo de la Secuencia Común Más Larga mediante división y
conquista y programación dinámica**

Asignatura: Análisis y Diseño de Algoritmos
NRC: 7909

Profesora: Mónica Villanueva
Fecha de entrega: 23 de junio de 2026

Estudiantes:

Jorge Buto Cumsille - 21.503.515-8
Álvaro Toro Herrera - 21.447.517-0

Índice

1. Introducción	3
2. Desarrollo	3
2.1 Algoritmos propuestos en pseudocódigo	3
2.1.1 Solución mediante división y conquista	3
2.1.2 Solución mediante programación dinámica	4
2.2 Complejidad del peor caso	5
2.2.1 Complejidad de la solución mediante división y conquista	5
2.2.2 Complejidad de la solución mediante programación dinámica	6
2.3 Traza de una instancia del problema	7
2.3.1 Traza mediante división y conquista	7
2.3.2 Traza mediante programación dinámica	10
2.4 Comparación de eficiencia, ventajas y desventajas	11
3. Conclusión	12
4. Referencias	12

1. Introducción

El presente informe analiza el problema de la Secuencia Común Más Larga, también conocida como SCM o LCS por su nombre en inglés Longest Common Subsequence. Este problema consiste en determinar la subsecuencia de mayor longitud que se encuentra presente en dos cadenas de texto, sin exigir que los caracteres seleccionados sean contiguos.

La SCM permite comparar cadenas de caracteres identificando similitudes entre ellas. Por esta razón, se utiliza frecuentemente como ejemplo clásico de aplicación de programación dinámica, debido a que el problema presenta subproblemas repetidos y una estructura óptima que permite construir la solución final a partir de soluciones parciales (Cormen et al., 2022).

El objetivo del trabajo es resolver el problema mediante dos técnicas de diseño de algoritmos: división y conquista y programación dinámica. La primera técnica plantea una solución recursiva pura que divide el problema en subproblemas más pequeños. La segunda técnica utiliza una tabla para almacenar resultados parciales y evitar el recálculo de subproblemas ya evaluados.

Para realizar la comparación experimental, se implementa un programa en Python que genera cadenas aleatorias con letras minúsculas del alfabeto inglés. Para cada tamaño de entrada n entre 5 y 15, ambos algoritmos se ejecutan utilizando exactamente los mismos textos, lo que permite realizar una comparación justa de sus tiempos de ejecución. Además, el programa genera un gráfico comparativo que permite observar el comportamiento empírico de ambas soluciones.

El código fuente se desarrolla considerando buenas prácticas de programación, tales como nombres representativos, organización mediante funciones y comentarios pertinentes, de acuerdo con las recomendaciones de PEP 8 (Python Software Foundation, 2025).

2. Desarrollo

2.1 Algoritmos propuestos en pseudocódigo

2.1.1 Solución mediante división y conquista

La solución mediante división y conquista se basa en comparar los últimos caracteres de ambas cadenas. Si estos caracteres coinciden, se consideran parte de una subsecuencia común y se suma una unidad al resultado del subproblema anterior. Si los caracteres no coinciden, se generan dos subproblemas: uno eliminando el último carácter de la primera cadena y otro eliminando el último carácter de la segunda cadena. El resultado corresponde al máximo valor obtenido entre ambas posibilidades.

Pseudocódigo:

Algoritmo SCM_Division_Conquista(X, Y, i, j)

Entrada:

X: primera cadena de texto

Y: segunda cadena de texto

i: cantidad de caracteres considerados de X

j: cantidad de caracteres considerados de Y

Salida:

Largo de la secuencia común más larga entre X[1..i] e Y[1..j]

Inicio

Si $i = 0$ o $j = 0$ Entonces

Retornar 0

Fin Si

Si $X[i] = Y[j]$ Entonces

Retornar $1 + \text{SCM_Division_Conquista}(X, Y, i - 1, j - 1)$

Si No

opcion1 = $\text{SCM_Division_Conquista}(X, Y, i - 1, j)$

opcion2 = $\text{SCM_Division_Conquista}(X, Y, i, j - 1)$

Retornar $\max(\text{opcion1}, \text{opcion2})$

Fin Si

Fin

Este algoritmo no utiliza memoria adicional para guardar resultados previos. Por ello, algunos subproblemas pueden ser calculados más de una vez durante la ejecución.

2.1.2 Solución mediante programación dinámica

La solución mediante programación dinámica utiliza una matriz DP. Cada celda $DP[i][j]$ representa el largo de la SCM entre los primeros i caracteres de la cadena X y los primeros j caracteres de la cadena Y. De esta manera, cada subproblema se resuelve una sola vez y su resultado queda disponible para cálculos posteriores.

Pseudocódigo:

Algoritmo $\text{SCM_Programacion_Dinamica}(X, Y)$

Entrada:

X: primera cadena de texto

Y: segunda cadena de texto

Salida:

Largo de la secuencia común más larga entre X e Y

Inicio

$m = \text{largo}(X)$

$n = \text{largo}(Y)$

Crear matriz DP de tamaño $(m + 1) \times (n + 1)$, inicializada en 0

Para i desde 1 hasta m Hacer

 Para j desde 1 hasta n Hacer

 Si $X[i] = Y[j]$ Entonces

$DP[i][j] = DP[i - 1][j - 1] + 1$

 Si No

$DP[i][j] = \max(DP[i - 1][j], DP[i][j - 1])$

 Fin Si

 Fin Para

Fin Para

Retornar $DP[m][n]$

Fin

La primera fila y la primera columna de la matriz se inicializan en cero, porque representan la comparación entre una cadena y una cadena vacía. Luego, la matriz se completa de forma ordenada, utilizando resultados previamente calculados.

2.2 Complejidad del peor caso

2.2.1 Complejidad de la solución mediante división y conquista

En el algoritmo mediante división y conquista, el peor caso ocurre cuando los caracteres comparados no coinciden con frecuencia. En ese escenario, cada llamada recursiva genera dos nuevas llamadas: una reduciendo la primera cadena y otra reduciendo la segunda cadena.

La recurrencia general puede expresarse como:

$$T(m, n) = T(m - 1, n) + T(m, n - 1) + O(1)$$

El término $O(1)$ representa el costo constante de comparar caracteres y calcular el máximo entre dos valores. Sin embargo, el costo dominante está dado por la expansión del árbol recursivo.

Como en esta tarea se consideran cadenas de igual largo n , se tiene que $m = n$. En el peor caso, la cantidad de llamadas recursivas crece exponencialmente. Una forma de acotar este crecimiento es considerar que la recursión puede expandirse hasta una cantidad proporcional a:

$$2^{(m+n)}$$

Como $m = n$:

$$2^{(n+n)} = 2^{2n} = 4^n$$

Por lo tanto, la complejidad temporal del algoritmo mediante división y conquista en el peor caso es:

$$O(4^n)$$

La complejidad espacial está asociada principalmente a la profundidad de la pila de recursión. Como en cada llamada se reduce al menos uno de los índices, la profundidad máxima es proporcional a $m + n$. Para cadenas de igual largo n , esto corresponde a:

$$O(n)$$

En consecuencia, el algoritmo por división y conquista es simple de formular, pero presenta un crecimiento temporal muy alto a medida que aumenta el tamaño de entrada.

2.2.2 Complejidad de la solución mediante programación dinámica

En el algoritmo mediante programación dinámica, se construye una matriz DP de tamaño $(m + 1) \times (n + 1)$. Cada celda de esta matriz se calcula una sola vez. Además, el cálculo de cada celda requiere tiempo constante, ya que solo se realiza una comparación y se consulta el valor de celdas previamente calculadas.

La cantidad total de celdas corresponde a:

$$(m + 1)(n + 1)$$

Al eliminar constantes y términos de menor orden, se obtiene:

$$O(m \cdot n)$$

Como en esta tarea ambas cadenas tienen largo n , se cumple que:

$$m = n$$

Por lo tanto:

$$O(m \cdot n) = O(n \cdot n) = O(n^2)$$

La complejidad temporal del algoritmo mediante programación dinámica es:

$$O(n^2)$$

La complejidad espacial también es:

$$O(n^2)$$

Esto se debe a que el algoritmo almacena una matriz con los resultados de los subproblemas. A cambio de este mayor uso de memoria, el algoritmo evita recalcular subproblemas repetidos y reduce considerablemente el tiempo de ejecución.

2.3 Traza de una instancia del problema

Para mostrar la resolución paso a paso, se utiliza la siguiente instancia con dos cadenas de largo 5:

X = "abcde"

Y = "abfde"

La secuencia común más larga entre ambas cadenas es:

"abde"

Por lo tanto, el largo esperado de la SCM es:

4

2.3.1 Traza mediante división y conquista

Se define $L(i, j)$ como el largo de la SCM entre los primeros i caracteres de X y los primeros j caracteres de Y .

Cadenas:

X = a b c d e

Y = a b f d e

La llamada inicial es:

$L(5,5)$

Como $X[5] = e$ e $Y[5] = e$, los caracteres coinciden:

$$L(5,5) = 1 + L(4,4)$$

Luego se evalúa $L(4,4)$. Como $X[4] = d$ e $Y[4] = d$, también coinciden:

$$L(4,4) = 1 + L(3,3)$$

Luego se evalúa $L(3,3)$. Como $X[3] = c$ e $Y[3] = f$, los caracteres no coinciden:

$$L(3,3) = \max(L(2,3), L(3,2))$$

Primera rama:

$L(2,3)$: compara b con f. No coinciden.

$$L(2,3) = \max(L(1,3), L(2,2))$$

$L(1,3)$: compara a con f. No coinciden.

$$L(1,3) = \max(L(0,3), L(1,2))$$

$$L(0,3) = 0$$

$L(1,2)$: compara a con b. No coinciden.

$$L(1,2) = \max(L(0,2), L(1,1))$$

$$L(0,2) = 0$$

$L(1,1)$: compara a con a. Coinciden.

$$L(1,1) = 1 + L(0,0) = 1$$

Por lo tanto:

$$L(1,2) = \max(0, 1) = 1$$

$$L(1,3) = \max(0, 1) = 1$$

$L(2,2)$: compara b con b. Coinciden.

$$L(2,2) = 1 + L(1,1) = 2$$

Entonces:

$$L(2,3) = \max(L(1,3), L(2,2))$$

$$L(2,3) = \max(1, 2) = 2$$

Segunda rama:

$L(3,2)$: compara c con b. No coinciden.

$$L(3,2) = \max(L(2,2), L(3,1))$$

$$L(2,2) = 2$$

$L(3,1)$: compara c con a. No coinciden.

$$L(3,1) = \max(L(2,1), L(3,0))$$

$$L(3,0) = 0$$

$L(2,1)$: compara b con a. No coinciden.

$$L(2,1) = \max(L(1,1), L(2,0))$$

$$L(1,1) = 1$$

$$L(2,0) = 0$$

Entonces:

$$L(2,1) = \max(1, 0) = 1$$

$$L(3,1) = \max(1, 0) = 1$$

$$L(3,2) = \max(2, 1) = 2$$

Ahora se resuelve $L(3,3)$:

$$L(3,3) = \max(L(2,3), L(3,2))$$

$$L(3,3) = \max(2, 2) = 2$$

Luego:

$$L(4,4) = 1 + L(3,3)$$

$$L(4,4) = 1 + 2 = 3$$

Finalmente:

$$L(5,5) = 1 + L(4,4)$$

$$L(5,5) = 1 + 3 = 4$$

Resultado final:

$$SCM = 4$$

Una secuencia común más larga posible es:

"abde"

En esta traza se observa que subproblemas como $L(2,2)$ y $L(1,1)$ aparecen más de una vez. Esta repetición permite explicar por qué el algoritmo recursivo puro tiene un mayor costo computacional.

2.3.2 Traza mediante programación dinámica

Para la misma instancia:

$X = \text{"abcde"}$

$Y = \text{"abfde"}$

Se construye una matriz DP donde las filas representan los prefijos de X y las columnas representan los prefijos de Y.

Tabla DP obtenida:

X \ Y	∅	a	b	f	d	e
∅	0	0	0	0	0	0
a	0	1	1	1	1	1
b	0	1	2	2	2	2
c	0	1	2	2	2	2
d	0	1	2	2	3	3
e	0	1	2	2	3	4

El llenado de la matriz permite observar lo siguiente:

- Al comparar a con a, se obtiene una coincidencia y el valor aumenta a 1.
- Al comparar b con b, se obtiene una nueva coincidencia y el valor aumenta a 2.
- Al comparar c con f, no existe coincidencia, por lo que se conserva el mejor valor previo.
- Al comparar d con d, el valor aumenta a 3.
- Al comparar e con e, el valor aumenta a 4.

El resultado final se encuentra en la última celda de la matriz:

$DP[5][5] = 4$

Resultado final:

SCM = 4

Una secuencia común más larga posible es:

"abde"

A diferencia de la solución recursiva pura, la programación dinámica calcula cada subproblema una sola vez. Los resultados quedan almacenados en la matriz DP y son reutilizados durante el llenado de las siguientes celdas.

2.4 Comparación de eficiencia, ventajas y desventajas

Para comparar el desempeño de ambos algoritmos, el programa desarrollado genera cadenas aleatorias con letras minúsculas del alfabeto inglés. Para cada tamaño de entrada n entre 5 y 15, se generan dos cadenas X e Y de largo n . Luego, ambas implementaciones se ejecutan utilizando exactamente las mismas cadenas.

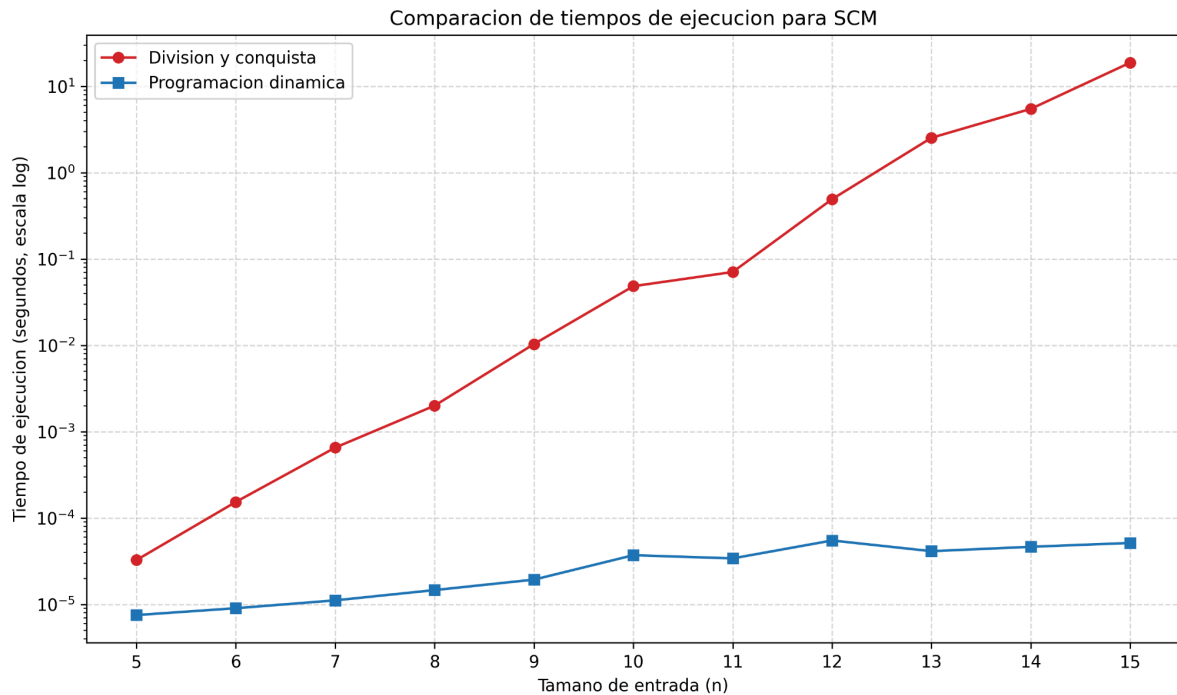


Figura 1. Comparación de tiempos de ejecución para la Secuencia Común Más Larga mediante división y conquista y programación dinámica.

En la Figura 1 se observa que la solución mediante división y conquista presenta un crecimiento mucho más pronunciado que la solución mediante programación dinámica. Esta diferencia se explica por la complejidad teórica de cada algoritmo. Mientras que el algoritmo recursivo puro puede alcanzar una complejidad temporal $O(4^n)$ en el peor caso, el algoritmo mediante programación dinámica presenta una complejidad temporal $O(n^2)$.

La escala vertical del gráfico se presenta en escala logarítmica. Esta decisión permite visualizar en una misma figura ambos algoritmos, debido a que sus tiempos de ejecución se encuentran en órdenes de magnitud muy distintos.

La solución mediante división y conquista tiene como ventaja principal su simplicidad conceptual. El algoritmo se deriva directamente de la definición recursiva del problema y permite representar claramente la división en subproblemas. Sin embargo, su principal desventaja es que recalcula muchas veces los mismos subproblemas, lo que provoca un aumento acelerado del tiempo de ejecución.

La solución mediante programación dinámica tiene como ventaja principal su eficiencia. Al almacenar los resultados de los subproblemas en una matriz, evita repetir cálculos y permite resolver el problema en tiempo cuadrático. Su principal desventaja es el uso de memoria adicional, ya que requiere almacenar una tabla de tamaño proporcional a n^2 .

Desde el punto de vista experimental, los resultados obtenidos son coherentes con el análisis teórico. La curva correspondiente a división y conquista crece rápidamente al aumentar n , mientras que la curva correspondiente a programación dinámica se mantiene significativamente más baja. Por esta razón, aunque ambos algoritmos entregan la misma

respuesta, programación dinámica resulta más adecuada para resolver el problema cuando el tamaño de entrada aumenta.

3. Conclusión

El análisis realizado permite comprobar que el problema de la Secuencia Común Más Larga puede resolverse mediante distintas técnicas de diseño de algoritmos, pero con diferencias importantes en eficiencia.

La solución mediante división y conquista permite resolver correctamente el problema utilizando una formulación recursiva directa. Sin embargo, su principal limitación es la repetición de subproblemas, lo que genera un crecimiento exponencial del tiempo de ejecución en el peor caso.

La solución mediante programación dinámica mejora considerablemente el desempeño del algoritmo, ya que almacena los resultados parciales en una matriz y evita recalcular subproblemas. Esto permite reducir la complejidad temporal a $O(n^2)$, considerando cadenas de igual largo n .

Los resultados experimentales obtenidos en el gráfico son consistentes con el análisis teórico. El algoritmo mediante división y conquista aumenta rápidamente su tiempo de ejecución, mientras que programación dinámica mantiene tiempos considerablemente menores. Por lo tanto, programación dinámica corresponde a la alternativa más eficiente y recomendable para resolver la Secuencia Común Más Larga en entradas de mayor tamaño.

4. Referencias

American Psychological Association. (s. f.). Reference examples. APA Style.
<https://apastyle.apa.org/style-grammar-guidelines/references/examples>

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). The MIT Press.
<https://mitpress.mit.edu/9780262046305/introduction-to-algorithms/>

Python Software Foundation. (2025). PEP 8 – Style Guide for Python Code. Python Enhancement Proposals. <https://peps.python.org/pep-0008/>

Villanueva, M., & Köhler, J. (2026). Tarea 4: Análisis y Diseño de Algoritmos [Enunciado de tarea]. Universidad Andrés Bello, Facultad de Ingeniería.